ASSIGNMENT 1

Comparative Study of Software Design Approaches

Project: AI Research Paper Discovery & Recommendation System

Item	Description
Project domain	Artificial Intelligence, Generative AI, Information Retrieval
Core capabilities	Academic paper discovery, hybrid search, reranking, RAG analysis, explainable recommendations
Proposed stack	Web frontend, FastAPI backend, scholarly APIs, metadata store, vector index and LLM services

This study describes a proposed design. A component should be described as implemented only after it has been built and tested.

1. Introduction

The AI Research Paper Discovery & Recommendation System helps students and researchers find relevant academic papers, understand their contributions, compare papers and explore possible research directions. A user submits a topic or natural-language question. The system interprets the query, retrieves candidate papers through keyword and semantic search, reranks results, and uses retrieval-augmented generation (RAG) to produce evidence-grounded summaries and explanations. Advanced features may include personalization, citation-based discovery, multi-paper comparison and research-gap hypotheses.

The system combines user interaction, paper ingestion, metadata management, retrieval, ranking, LLM-based analysis and evaluation. The design should separate these responsibilities so that each can be developed, tested and improved independently.

2. Software design approaches

2.1 Structured / Function-Oriented Design

This approach decomposes the system into functions and sub-functions, often shown through data-flow diagrams and structure charts. The main function, process research request, can be divided into validate query, retrieve papers, rank results, analyze evidence and return recommendations.

Advantages: clear control flow, easy-to-follow hierarchy, suitable for a structure chart and a well-defined pipeline. **Disadvantages:** shared data may become tightly coupled; changes to a large function can affect other functions; domain entities are less naturally represented. **Suitability:** high for describing the workflow, but not ideal as the only design technique.

2.2 Object-Oriented Design (OOD)

OOD organizes code around objects that combine state and behaviour. Relevant objects include Paper, Author, ResearchQuery, UserProfile, Citation, SearchResult and Recommendation.

Advantages: models academic entities naturally, encourages encapsulation and reuse, and supports unit testing. **Disadvantages:** excessive classes or inheritance can create needless complexity; AI workflow orchestration still needs explicit design. **Suitability:** high for domain models and service boundaries.

2.3 Layered Architecture

A layered design separates presentation, application/orchestration, domain or AI services, and data/integration. The frontend calls FastAPI; application services coordinate query understanding, retrieval, reranking and RAG; adapters access databases, vector search and scholarly APIs.

Advantages: separation of concerns, easier testing and debugging, and replaceable storage/provider adapters. **Disadvantages:** requests pass through multiple layers and poorly designed layers can duplicate logic. **Suitability:** very high as the overall architecture.

2.4 Component-Based / Modular Design

The system is split into components with defined interfaces: ingestion, query understanding, hybrid retrieval, reranking, RAG generation, recommendation and evaluation. Each component has a focused contract, such as retrieve_candidates(query, filters) or generate_analysis(question, evidence).

Advantages: independent development and testing, replaceable model providers, easier experimentation and parallel work. **Disadvantages:** interfaces require discipline; integration and version compatibility take effort; too many tiny modules can be confusing. **Suitability:** very high because retrieval and GenAI stages need separate evaluation and improvement.

2.5 Data-Centred Design

This approach emphasizes data structures, data quality and access patterns. The project stores paper metadata, abstracts, extracted text, embeddings, citation links, optional user preferences and evaluation judgements.

Advantages: encourages deduplication, stable identifiers, traceability and consistent indexing. **Disadvantages:** schema changes may affect several modules; weak data modelling limits future features; external source quality remains a concern. **Suitability:** high because recommendations depend on accurate metadata and reliable paper links.

2.6 Microservices Architecture

Microservices deploy capabilities such as ingestion, retrieval and LLM analysis as separate network services.

Advantages: independent deployment and scaling, technology flexibility and possible fault isolation. **Disadvantages:** higher deployment, monitoring, security and networking overhead; distributed debugging and consistency are harder. **Suitability:** low to medium for the first version. A modular monolith is more practical until scale or team structure justifies separate services.

3. Comparative evaluation

Approach	Main strength	Main limitation	Fit
Function-oriented	Clear functions and control flow	Can couple shared data	High for workflow/chart
Object-oriented	Models papers, users and results	Can become class-heavy	High for domain model
Layered architecture	Separates UI, services and data	Layer hopping / duplicated logic	Very high overall
Component-based	Replaceable, testable AI stages	Needs stable interfaces	Very high
Data-centred	Metadata quality and traceability	Schema changes can ripple	High
Microservices	Independent deployment/scaling	Operational complexity	Not for initial version

4. Recommended approach

Use a **layered, modular architecture**, supported by object-oriented domain models and a function-oriented structure chart. This combines complementary methods: layers define major boundaries, modules isolate AI capabilities, objects represent scholarly entities, and the structure chart explains the hierarchy.

Layer	Responsibilities	Examples
Presentation	Query form, filters, results, paper detail and follow-up UI	React / Next.js
Application / orchestration	Validate requests and coordinate pipeline stages	FastAPI endpoints and services
Domain / AI services	Query understanding, hybrid search, reranking, RAG and recommendations	Python modules
Data / integration	Paper APIs, metadata, extracted text and vector index	OpenAlex/arXiv/Semantic Scholar adapters, database, vector DB
Evaluation / operations	Measure relevance, grounding, latency and errors	Offline test scripts, logs and metrics

5. Design guidelines

• Define stable contracts: query understanding returns a structured query; retrieval returns paper IDs and scores; reranking returns an ordered list; RAG receives evidence with source identifiers.

• Keep canonical metadata separate from extracted text and embeddings. Deduplicate papers using stable identifiers such as DOI or source IDs.

• Ground summaries and comparisons in retrieved evidence. Include source links and section/page details where available. Label generated research directions as hypotheses unless the evidence directly supports them.

• Evaluate BM25, dense retrieval, hybrid retrieval and reranking independently on the same fixed query set.

• Begin with a modular monolith. Adopt microservices only when real deployment, scaling or team needs justify their cost.

6. Conclusion

A layered, modular design offers the best balance of clarity, testability and extensibility for this project. Object-oriented models manage scholarly entities, data-centred practices protect metadata quality, and function-oriented decomposition communicates the workflow. Microservices remain a possible future option rather than a first-version requirement.

References / further reading

1. Sommerville, I. (2015). *Software Engineering* (10th ed.). Pearson.

2. Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design.* Prentice Hall.

3. Bass, L., Clements, P., & Kazman, R. (2021). *Software Architecture in Practice* (4th ed.). Addison-Wesley.

4. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software.* Addison-Wesley.